

Reviewer Pack — Minimal Rank of tropicalized Homology Groups over Boolean Fu...

Entry 3c38b1d67d89 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 03:36:39 UTC

Minimal Rank of tropicalized Homology Groups over Boolean Functions vs Monotone Circuit Complexity for XOR

Entry ID: 3c38b1d67d89 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 03:36:39 UTC

1. Conjecture

Field A (mathematical branch): Homological Algebra (Tropicalized Homology) **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity

Statement:

[‘For any n -ary boolean function f , let $H(f)$ be its tropicalized homology group with coefficients in the Boolean ring. The conjecture states that for all monotone circuits C computing f , if C has at most n layers, then the minimal rank of $H(f)$ is lower-bounded by the number of gates in C .’, ‘Equivalently, for every boolean function f with tropicalized homology group of minimal rank r , there exists a monotone circuit computing f that has at most $r(n+1)$ gates.’, ‘Moreover, this relation holds with high probability when considering random boolean functions.’]

Rationale (proposer’s reasoning):

[‘Tropicalized homology provides an algebraic invariant for boolean functions, offering a potential new perspective on circuit complexity. The conjecture aims to establish a connection between the geometric structure of boolean functions and their computational complexity.’, ‘If true, this link could lead to new algorithms for monotone circuits or provide insights into the inherent difficulty of computing certain boolean functions.’, ‘The construction of tropicalized homology groups is computationally feasible, making it possible to test the conjecture on instances of size $n \leq 40$ within the required time frame.’]

Taxonomy category: SOS_DEGREE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 584efeb184e3551e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all n -ary boolean functions f and at least 80% of the generated monotone circuits C with at most n layers, the number of gates $|\text{Gate}(C)|$ satisfies $|\text{Gate}(C)| \leq r(H(f))(n+1)$, where $r(H(f))$ is the minimal rank of the tropicalized homology

group $H(f)$ over the Boolean ring. The conjecture is falsified if any seed produces a circuit C with $|\text{Gate}(C)| > r(H(f))(n+1)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropicalized homology" AND "monotone circuit complexity" - "minimal rank of Homology Groups" near "boolean function" AND "circuit complexity" - "Boolean ring" INTC "Homological Algebra" AND "Monotone Circuit Complexity"

Top relevant hits considered: - [http://arxiv.org/abs/1609.04274v2] Polymorphisms and Circuit Complexity - [http://arxiv.org/abs/2511.21738v4] On the Incompressibility of Truth With Application to Circuit Complexity - [http://arxiv.org/abs/0811.0699v1] A Note on the Inversion Complexity of Boolean Functions in Boolean Formulas - [http://arxiv.org/abs/1608.03437v1] Coherent spaces, Boolean rings and quantum gates - [http://arxiv.org/abs/1502.07220v2] Groebner basis in Boolean rings is not polynomial-space - [http://arxiv.org/abs/1707.07783v1] Primary Decomposition in Boolean Rings

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def tropicalized_homology_group(f):
        n = int(math.log2(len(f)))
        homology = [[0] * (n + 1) for _ in range(n + 1)]
        for i in range(n + 1):
            for j in range(n + 1):
                if i == j:
                    homology[i][j] = sum(f[k] for k in range(2**n) if bin(k).count('1') == i)
        return homology

    def min_rank(homology):
```

```

n = len(homology) - 1
rank = 0
for _ in range(n + 1):
    pivot_row = next((i for i in range(rank, n + 1) if homology[i][rank] != 0), None)
    if pivot_row is None:
        break
    homology[pivot_row], homology[rank] = homology[rank], homology[pivot_row]
    for j in range(n + 1):
        if j == rank:
            continue
        factor = homology[j][rank] / homology[rank][rank]
        for k in range(n + 1):
            homology[j][k] -= factor * homology[rank][k]
    rank += 1
return rank

def construct_monotone_circuit(f, n):
    # Placeholder function to construct a monotone circuit
    # This is a dummy implementation and should be replaced with actual logic
    return [random.randint(2, n) for _ in range(len(f))]

def gate_count(circuit):
    return sum(circuit)

n = random.randint(5, 40)
f = generate_boolean_function(n)
homology = tropicalized_homology_group(f)
r = min_rank(homology)

circuits = [construct_monotone_circuit(f, n) for _ in range(30)]
conjecture_holds = all(gate_count(circuit) <= r * (n + 1) for circuit in circuits)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "Gate Count",
    "metric_value": sum(gate_count(circuit) for circuit in circuits),
    "instances_tested": len(circuits),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 150, 5))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = (sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))**0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):

```

```

    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot verify the conjecture's support or falsification under the pre-registered conditions. | next: None

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12075
2	propose	ollama_remote	glm4:latest	0	0	11714
3	preregistration	ollama_remote	glm4:latest	0	0	6166
4	novelty	ollama_remote	glm4:latest	0	0	4751
5	novelty	ollama_remote	glm4:latest	0	0	6051
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14455
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13700
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9176
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10750
10	judge	ollama_remote	glm4:latest	0	0	17121

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 105959 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in [research/audit/3c38b1d67d89.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3c38b1d67d89.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/3c38b1d67d89.tar.gz` (if generated)